Model-Based Agentic Software Engineering

James C. Davis
davisjam@purdue.edu
Purdue University
West Lafayette, IN, USA

Kelechi G. Kalu
kalu@purdue.edu
Purdue University
West Lafayette, IN, USA

Huiyun Peng
peng397@purdue.edu
Purdue University
West Lafayette, IN, USA

Parth Vinod Patil
parthvp@amazon.com
Amazon Robotics
WestBorough, MA, USA

Abstract

Coding agents increase implementation capacity without automatically making project intent, system structure, or acceptance evidence explicit. As implementation becomes abundant relative to engineering judgment, the scarce work shifts toward choosing useful abstractions, producing evidence, and determining which obligations govern acceptance. Existing workflows address parts of this gap through larger prompts, repository retrieval, or per-change review, but still require agents and engineers to reconstruct consequential properties.

As an alternative, we present Model-Based Agentic Software Engineering (MAGE). MAGE is a framework and a theory for building trustworthy autonomy from commodity intelligence. MAGE addresses a representation problem and an authority problem: it externalizes the smallest purposeful representation needed to answer an engineering question, then gives settled obligations proportionate authority through constraints, sensors, validators, and gates. It keeps uncertain intent open and turns recurring reconstruction and judgment into durable engineering structure that later work can inherit. We developed MAGE from a longitudinal case and refined it through six independently reported industrial accounts. Across these sources, MAGE explains how externalized knowledge, bounded action, independent evaluation, and retained human authority can compose into a governed engineering environment, and proposes tests of when that environment turns commodity intelligence into durable engineering progress.

CCS Concepts

• **Software and its engineering** → **Software development methods**; • **Computing methodologies** → *Artificial intelligence*.

Keywords

agentic software engineering, software models, representation engineering, coding agents, engineering governance

1 Introduction

Coding agents are moving software engineering toward a model in which developers specify goals while agents plan, implement, test, and optimize software [12, 22]. This shift raises a central question of *assurance at velocity*: *how can we trust systems that can act with limited human intervention?* As coding agents increase the rate at which software changes can be produced, human specification, understanding, and validation can become the limiting factors in the development process [6, 18, 23].

Current approaches primarily improve agents' access to information through retrieval [17], memory [11], tools [35], and harnesses [30, 31]. However, more context does not necessarily make engineering properties explicit. An agent asked whether a change preserves a system boundary may still need to reconstruct that boundary from source code, tests, configuration, and history. A human asked to validate the same change may face the same reconstruction problem. As change volume increases, repeatedly reconstructing and adjudicating these properties becomes a bottleneck for both autonomous reasoning and human oversight. The challenge is therefore not only to provide more context, but to make consequential engineering properties explicit so they can be reasoned about, evaluated, and governed without repeated reconstruction.

To meet this challenge, we present *Model-Based Agentic Software Engineering (MAGE)*, a theory of trustworthy agentic software engineering centered on the governed engineering environment.[1] MAGE addresses two problems: representation and authority. Modeling makes consequential engineering knowledge and intent explicit so humans and agents can reason about larger properties without repeatedly reconstructing them from code. Alignment gives important engineering obligations authority through governance mechanisms (distinguished into constraints, sensors, validators, and gates). When failures expose missing knowledge or unenforced obligations, MAGE turns those lessons into durable engineering structure that later work can inherit. Together, these mechanisms allow greater agent autonomy while reducing the reconstruction and repeated human judgment needed to govern it.

We developed MAGE in two stages. First, a longitudinal case study of a single project, DocAble, identified underlying mechanisms and their temporal ordering. This study traced how representations, controls, and recurring judgment evolved within a single system. Second, comparative reconstructions of six independently reported approaches in industry examined whether related structures emerged under different organizational and technical pressures and identified their scope conditions. Together, these analyses supported the development and refinement of MAGE as a theory of how these structures interact to enable reliable agentic engineering.

This paper contributes:

- *MAGE, a theory of trustworthy agentic software engineering* that explains how explicit engineering knowledge and authoritative obligations compose into a governed engineering environment.
- *empirical grounding and refinement of the theory* through a longitudinal case study of DocAble and comparative reconstruction of six independently reported industrial systems, exposing recurring mechanisms, alternative realizations, and scope conditions.
- *A falsifiable research agenda* for testing when governed engineering environments support human judgment and convert agentic capacity into durable engineering progress.

[1] For an expanded treatment of these topics, see our MAGE book at https://davisjam.github.io/model-based-agentic-software-engineering/book/mage-book.pdf.

Relevance to JAWs. We will develop this work into a full TSE manuscript that clarifies the theory and strengthens its methodological and empirical grounding. We will also elaborate MAGE's relationship to established traditions in software engineering, systems engineering, formal methods, and AI. JAWs provides an opportunity to obtain early community feedback on a theory.

2 Background

MAGE builds on governance in software engineering and two established traditions: engineering approaches for making system knowledge explicit and enforceable, and AI approaches for representing, retaining, and acting on state. We first define governance in the operational sense used in this paper, then summarize the two traditions that MAGE composes.

2.1 Governance in Software Engineering

Governance describes the problem of making consequential engineering decisions hold as a software system changes: what work may proceed, who may authorize it, what evidence a change must produce, and what conditions admit it. Conventional software engineering provides many mechanisms that serve these purposes, including requirements and architecture, ownership and permissions, review and test policies, continuous-integration gates, release procedures, and incident learning [33]. These mechanisms are distributed across engineering practice rather than organized around a common account of how autonomous work should be governed.

Agentic software engineering makes this problem more acute. Recent work has begun to explore governance mechanisms for autonomous agents, including policies, runtime controls, and human-agent authority boundaries [1, 13, 15]. *Ex ante* mechanisms can encode known obligations before work begins [15], while *ex post* governance responds when failures expose missing knowledge or unstated obligations; governance conversion makes those lessons durable for later work [6]. As implementation outpaces per-change review, the unresolved question is how these functions should be realized in an engineering environment capable of governing autonomous work at scale.

2.2 Engineering Traditions

Software and systems engineering have long addressed the problem of making consequential knowledge explicit and actionable. Model-based engineering externalizes structure and behavior [27]; languages and compilers encode properties through types, interfaces, and intermediate representations [24]; formal methods connect specifications to verification [4]; and systems and safety engineering establish boundaries, hazards, assurance arguments, and independent controls. Across these traditions, a representation is a purposeful reduction: a graph, state machine, contract, or quantitative model preserves what a question needs while suppressing other detail. It reduces repeated reconstruction and creates a stable object for reasoning, checking, transformation, and traceability.

2.3 AI Traditions

AI provides complementary mechanisms for representing, retaining, and acting on state. Knowledge representation captures facts and rules [21]; memory carries information across episodes [11]; planning and structured reasoning organize intermediate state [32, 35]; and tools connect reasoning to repositories, tests, simulators, permissions, and feedback [26]. In coding agents, context and retrieval shape what is available [17], while harnesses, workflows, and tools shape what the agent can do and what evidence it returns [31, 34]. The "Printer" metaphor captures the resulting division of labor: implementation becomes abundant, while specifying intent, selecting abstractions, producing evidence, and determining acceptance remain engineering responsibilities.

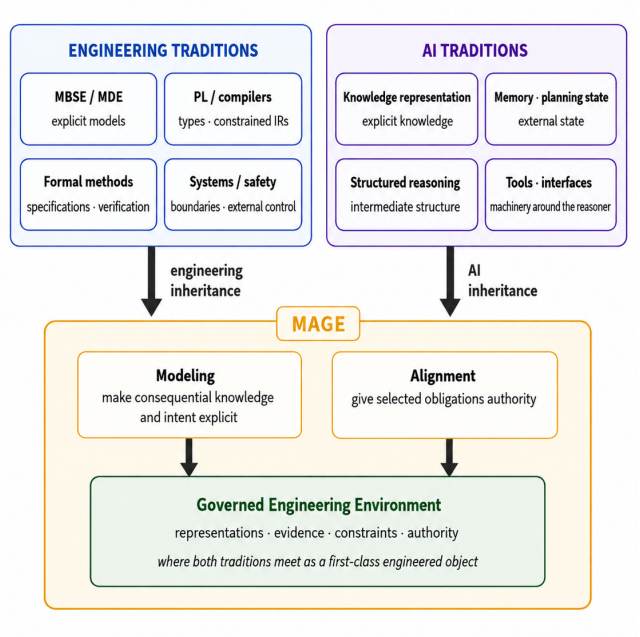

Figure 1: Conceptual foundations of MAGE. The framework combines engineering mechanisms for explicit representation, verification, and control with AI mechanisms for external state, structured reasoning, and tool-mediated action. Modeling and Alignment connect these traditions in the governed engineering environment.

The paper's central distinction. The operational account above explains what governance must accomplish; the two traditions supply means for doing so. MAGE combines them in the *Governed Engineering Environment (GEE)*: an engineered environment that surrounds autonomous implementation with explicit knowledge and enforceable obligations. Its novelty is their composition, not any one model or control. *Modeling* makes consequential intent and system knowledge explicit in purposeful representations. *Alignment* gives selected properties operational authority through constraints, sensors, validators, and gates. The GEE joins the two so engineering knowledge can guide, constrain, and evaluate autonomous action.

3 Motivation

Assurance at velocity cannot be inferred from agent capability alone. Coding agents expand implementation capacity, while the preceding account of governance shows that explicit knowledge and enforceable obligations reside in the surrounding engineering

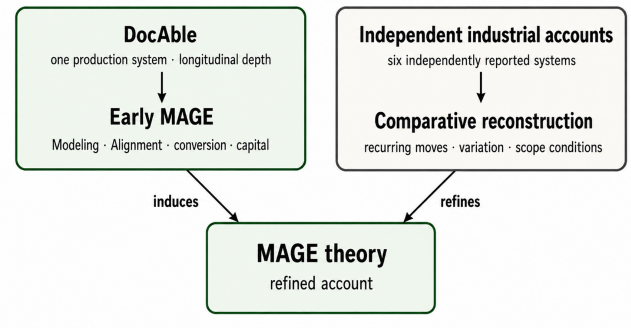

Figure 2: Theory-development design. The longitudinal DocAble case induces initial MAGE constructs, while comparative reconstruction across six independent industrial accounts refines their recurring mechanisms, variation, and scope conditions.

environment. An empirical account of outcomes should therefore distinguish the agentic capability made available from the engineering environment through which it is used. An adoption indicator marks access to the former but can leave the latter unspecified.

For example, He *et al.* examined the relationship between Cursor AI adoption, development velocity, and software quality in open-source projects [10]. Their design captures when the tool enters a project, but not a common software-engineering method or environment through which it is operated. This distinction is familiar from studies of open-ended technologies, whose effects depend on how their capabilities are enacted in practice [16, 20]. Generative AI is especially open-ended: developers may use the same technology for completion, debugging, review, testing, optimization, or autonomous implementation, under different task boundaries, context, oversight, and acceptance criteria. Projects placed in the same "adopted" condition may therefore receive materially different interventions, limiting what adoption alone can explain.

We believe that the missing empirical object is the *governed engineering environment*: the representations, evidence, constraints, procedures, and authority through which agentic work is reasoned about, constrained, and admitted. Characterizing this environment makes it possible to distinguish access to a general-purpose capability from a reproducible software-engineering intervention. Before its effects can be tested, however, its constructs and mechanisms must be identified. This need motivates the exploratory theory-building design described next.

4 Methodology

This section describes how we developed MAGE (Figure 2). We use longitudinal evidence from DocAble to identify mechanisms and develop provisional constructs, then compare these constructs with six independently reported industrial systems to examine alternative realizations.

4.1 Research Design and Evidence

We use an exploratory, interpretive theory-building design with two sources of evidence: a longitudinal case of DocAble and comparative reconstructions of six independently reported industrial accounts. The DocAble case provides the empirical starting point and temporal record from which the initial mechanisms and constructs were developed. The industrial accounts provide variation for examining whether related structures recur under different organizational and technical conditions.

DocAble. DocAble is a document-accessibility system developed by the lead author in response to recent regulatory changes affecting public universities under Title II of the Americans with Disabilities Act [29]. DocAble was developed over approximately 20 weeks of full-time work using coding agents as the primary implementation workforce. The author exhausted 3–4 Claude Max 20x accounts during most weeks of the project. By the end of the observation period, the system comprised approximately 540,000 lines of production code and 1.6 million lines of supporting governed-environment infrastructure. Further details of the system and its evolution are available in the MAGE book [5].

Our analysis builds on the case study initially reported in [6], which reconstructed engineering episodes from contemporaneous field notes and repository history. The present study does not independently analyze those field notes; it uses that case analysis together with the subsequent evolution of the DocAble repository to develop and refine MAGE.

Comparative sample of industry accounts. The comparative sample is purposive and comprises first-party accounts from Cloudflare, Spotify, Shopify, Docker, Siemens, and Zenseact. We selected accounts that describe autonomous or agentic software work with sufficient architectural or organizational detail to reconstruct relevant representations, action boundaries, evaluation mechanisms, and retained human authority. These accounts provide variation in engineering pressures, platforms, organizational settings, and approaches to autonomous work. Table 1 previews the source claims and MAGE interpretations examined in the comparative stage.

4.2 Analysis and Theory Development

DocAble theory development. The initial analysis of the DocAble record, reported in the Cheap Code study [6], provides the empirical starting point for MAGE. That analysis reconstructed salient episodes through their trigger, interpretation, response, immediate outcome, and effects on later work, and produced the failure-to-governance process model. Here, that analysis and the subsequent DocAble record provide an empirical seed rather than a complete derivation of the theory.

Over the following two months, the author team developed this account iteratively. We proposed candidate constructs and relationships, applied them to recurring engineering questions, and revised them when we encountered contradictions, missing mechanisms, or scope conditions. This process expanded our earlier emphasis on ex-post governance [6] to include models that make consequential knowledge available before work begins and the role of accumulated failure knowledge in Alignment. It yielded provisional constructs for Modeling, Alignment, governance conversion, and engineering capital.

We also challenged and refined the emerging account through practitioner conversations, public writing and discussion, and feedback on talks at organizations including Argonne National Laboratory and IBM Research. These engagements exposed unclear terms,

Table 1: Comparative reconstruction of six independent industrial accounts. "Observed structure" summarizes selective first-party source claims; "MAGE lens" is our interpretation. The Siemens source describes prototypes rather than production deployment.

Account and pressure	Observed structure	MAGE lens
Cloudflare [25] Institutional standards	Structured requirements with stable identities, progressive retrieval, and separation of approved guidance from enforced obligations.	Modeling institutional knowledge plus local Alignment.
Spotify [28] Fleet migration	Persistent service and lineage information, fleet targeting, build/test feedback, and managed admission.	Platform-level knowledge, evaluation, and admission authority.
Shopify [19] Organizational knowledge	Reproducible repository substrate, on-demand skills, durable sessions, searchable work, and useful sessions carried into inherited defaults.	Externalized knowledge and governance conversion through platform defaults.
Docker [7] Delegated authority	Bounded roles, tools, and sandboxes; separate producing and reviewing agents; human retention of the merge decision.	Alignment through action boundaries, independent review, and retained authority.
Siemens [2] Model-first engineering	Task agents operating over persistent engineering models, simulation, and expert-defined workflows.	A Modeling-first configuration with an explicit evaluation surface.
Zenseact [8] Distributed ownership	Centrally governed platform with team-owned agents and domain knowledge, using progressive disclosure of relevant skills and tools.	Federated authority and task-relevant context delivery.

counterexamples, and alternative realizations. They informed theory development but are not treated as empirical observations or independent validation.

Comparative analysis. The comparative stage examined recurrence, alternative realizations, and scope conditions across the industrial accounts. Each account was analyzed using a common frame: engineering pressure; externalized representation; action boundaries; evidence and evaluation; admission authority; mechanisms through which later work inherits prior knowledge; and scope conditions. We kept direct source claims separate from MAGE interpretations and recorded the analysis in a case-by-construct matrix linking source material to the refined theory. Ambiguous and negative observations were retained during comparison.

Across the industrial accounts, we observed recurring structures: knowledge is externalized, action passes through bounded tools or roles, generation is separated from evaluation, and consequential authority remains human where the decision is not adequately mechanized. The accounts arise from different engineering pressures rather than a shared MAGE adoption program, and several do not establish model correspondence, longitudinal adaptation, or outcome measures. The comparison therefore supports claims of recurrence and variation, not causal effectiveness.

4.3 Empirical Grounding and Validity

DocAble grounding. The DocAble record provides quantitative grounding for the theory-development account above. During the 20-week build, 6–8 agents routinely worked in parallel, sustaining approximately 200 commits per day and 1,000 per week. This rate exceeded what one engineer could review directly, and quality degraded as the system grew. The project responded by shifting increasing amounts of engineering work into durable models, checks, and controls.

This hardening is visible in the composition of the repository. Support apparatus—tests, models, orchestration, documentation, and governance tooling—grew from 0.85× production source after the prototype to approximately 3× in the mature snapshots, peaking at 3.68× during hardening. Figure 3 shows this trajectory.

Repository measures show several forms of hardening. Project-specific lint files grew from 0 to 747 and gate scripts from 0 to 102; 208 commits paired a fix with a lint intended to catch recurrence. Derived checks caught six instances of a previously identified class

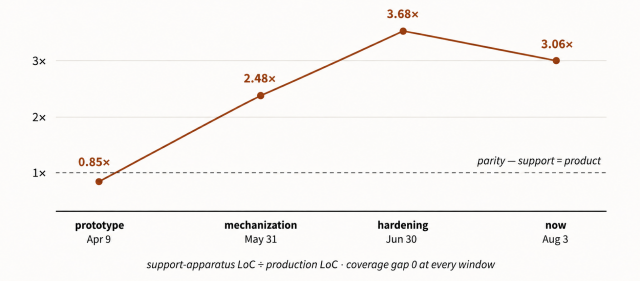

Figure 3: Evolution of DocAble's support ratio across four repository snapshots. Support-apparatus source grew from 0.85× production source after the prototype to approximately 3× in the mature snapshots, peaking at 3.68× during hardening.

of model–code drift, with no observed recurrence of that mechanically decidable class across 56 subsequent feature implementations. Across a nine-stage modeling sequence, the proportion of unmodeled implementation elements decreased from 56% to 7.89%.

Validity. The industrial accounts add variation across organizations and engineering settings, with less visibility into internal failures and evolution. Together, these sources support theory development, comparative refinement, and analytic generalization within the stated scope conditions. They do not support population estimates or causal claims about MAGE's effectiveness.

5 MAGE Theory

MAGE's core claim is that durable engineering progress from agentic work depends not only on agentic capacity, but also on the quality of the engineering environment in which agents operate. This environment make consequential properties answerable and give selected obligations sufficient authority. This section defines MAGE's explanatory model, introduces its core constructs and relationships, and shows how they form feedback loops that distinguish MAGE from a collection of agent tools or governance checks.

5.1 MAGE's Explanatory Model

Agentic capacity becomes durable engineering progress only when the environment makes consequential properties answerable and

gives selected obligations authority. Because agentic capacity amplifies its environment, a strong environment turns that capacity into durable throughput, while a weak one produces churn.

MAGE captures these functions through two mechanisms:

- **Modeling** externalizes consequential engineering knowledge and intent into explicit, structured models that engineers and agents can reason through, instead of reconstructing meaning from implementation for every question.
- **Alignment** gives selected obligations authority through constraints, sensors, validators, and gates that influence agent behavior or determine whether work is accepted.

Modeling is the purposeful reduction of system detail around a specific engineering question. A dependency graph can represent architectural relationships, a state machine can specify permitted transitions, a contract can specify interface obligations, and a quantitative model can characterize relevant system behavior. Alignment then connects selected properties to mechanisms that can affect outcomes, such as permission boundaries, tests, admission rules, or deployment gates. Together, Modeling and Alignment determine which properties are both formally representable and operationally consequential within the engineering environment.

Positioning. MAGE operates at the level of the engineered environment around autonomous implementation. It complements existing approaches by distinguishing two functions: *Modeling* represents consequential properties, while *Alignment* gives selected properties force through evaluation and control mechanisms.

Scale and assurance strength are independent choices. An individual engineer may apply formal verification to a consequential protocol, while a large organization relies primarily on documentation, testing, runtime evidence, and human judgment. Broad organizational adoption does not require maximal formalization, just as strong assurance does not require organization-wide adoption. MAGE therefore spans a range from lightweight representations of intent and selective checks to formal specifications and trustworthy derivation. Across these settings, the underlying engineering problem remains the same: determining what must be represented, what evidence is sufficient, which obligations should have operational authority, and where human judgment remains necessary.

The theoretical contribution lies in this composition of representations and controls over time. Prior knowledge can structure work before execution; representations and controls can guide work during execution; and recurring judgment can be captured as reusable structure for subsequent work. In this way, MAGE extends the earlier middle-range theory [6] from reactive governance to an engineering environment that accumulates and reuses judgment.

5.2 Constructs and Relationships

The theory centers on a simple mediation claim: *agentic capacity affects realized performance through the governed engineering environment.* The environment does more than constrain work. It determines which properties can be made answerable and which obligations can be given authority. The constructs below define the terms of this mechanism.

Agentic capacity is the ability to generate, inspect, and modify software through autonomous or semi-autonomous work. It expands the amount and scope of work that can be attempted, but does not by itself produce durable engineering progress. The *task frontier* is the set of tasks and system scopes that can be undertaken with acceptable risk and human effort. The *governed engineering environment* comprises the representations, evidence, constraints, procedures, and authority through which engineering work is understood, constrained, and admitted.

Environment quality has multiple dimensions. *Representation quality* concerns whether the environment exposes the properties relevant to the engineering question. *Obligation coverage* concerns whether important obligations are stated and evaluated during the engineering process. *Coherence* concerns whether representations, implementation, evidence, and controls remain mutually consistent. *Economy* concerns whether the benefits of this structure justify its construction, delivery, and maintenance costs. More artifacts do not necessarily produce a higher-quality environment. The relevant criterion is whether the environment makes the right properties answerable and the right obligations enforceable at acceptable cost.

A central source of poor environment quality is the *semantic gap*: the distance between where an engineering obligation arises and where the environment has sufficient meaning, evidence, and authority to determine whether it is satisfied. When this gap is large, engineers must repeatedly reconstruct context, exercise judgment, or defer decisions to a later boundary where the relevant system state is observable. Modeling reduces the gap by making missing properties explicit. Alignment reduces it by giving obligations operational force through constraints, validators, and gates.

Realized performance captures durable throughput and the human attention required to sustain work, rather than implementation volume alone. When attempted work exceeds what the environment can make answerable or enforceable, a mismatch creates *governance pressure. Structural diagnosis* identifies the source of this mismatch, such as a local defect, missing representation, weak authority, stale correspondence, an unbridged semantic gap, or an uneconomical mechanism. *Governance adaptation* responds by modifying the environment's models, evidence, procedures, or controls.

Finally, *engineering capital* is durable structure that reduces the cost or uncertainty of future work through reuse. Recurring judgment becomes engineering capital when it is converted into a model, procedure, or mechanism whose expected future value exceeds the cost of constructing and maintaining it.

5.3 Relationships and Propositions

These constructs are linked by a directional logic rather than a single fitted causal model. Agentic capacity increases the amount of work that can be attempted, while the governed environment mediates whether that work becomes durable performance. Modeling contributes representation quality; Alignment contributes authority; and their combination shapes environment quality. When governance adaptation converts recurring judgment into economically durable structure, it builds engineering capital and expands the task frontier.

The constructs therefore support four directional propositions.

P1: Environment fit moderates capacity. Increasing agentic capacity should produce more durable throughput when the governed environment fits the work, and more churn, escaped failure, or repeated intervention when it does not.

P2: Representation leverage increases with reasoning burden. A task-relevant, faithful representation reduces the reconstruction needed to answer an engineering question, with larger gains as system state grows. The claim fails when maintenance and interpretation costs exceed the reconstruction saved.

P3: Authority expands the consequential surface. Alignment can make an obligation consequential without a rich system model, while Modeling can make additional system-level obligations legible to Alignment. Their effects are complementary, not interchangeable, and the semantic gap sets how much authority must be deferred or reconstructed rather than enforced in place.

P4: Engineering capital amortizes judgment locally. Durable structure should reduce subsequent reconstruction, repeated judgment, rework, or risk on the engineering surfaces that inherit it. Its return should diminish when the structure creates conflicts or costs more to maintain than it saves.

5.4 Dynamics

Figure 4 summarizes the conceptual chain underlying these dynamics. Prior engineering knowledge can bootstrap a governed environment before failures occur. Agentic capacity and attempted work then operate through this environment and shape realized performance. When a task exceeds the environment's ability to make relevant properties answerable or obligations enforceable, the semantic gap becomes visible as governance pressure: the obligation exists, but the environment lacks the representation, evidence, authority, or decision boundary needed to act on it.

Structural diagnosis distinguishes local defects from the broader forms of environment weakness defined above. Governance adaptation responds by changing the models, procedures, evidence, or controls available to future work. In this sense, MAGE treats recurring performance problems not only as implementation errors, but also as potential failures of environment design.

The resulting dynamics contain two distinct feedback paths. The *pressure-and-adaptation* path is balancing: governance pressure prompts changes that reduce the mismatch between attempted work and the environment. The *capability-amplification* path is reinforcing: a stronger environment supports larger tasks or greater concurrency, expanding the task frontier and creating new reasoning and governance demands. Engineering capital can therefore expand the task frontier, but it can also depreciate as the system, obligations, or maintenance costs change.

The model is therefore a directional theory of these relationships. It predicts that durable gains depend on whether the environment makes the relevant properties answerable, gives selected obligations sufficient authority, and does so economically enough for the resulting structure to persist.

6 Application of MAGE

In this section, we translate the theory into a practical method. We show how an engineering question determines the appropriate starting point, how Modeling and Alignment apply across project states and scales, and how the same mechanisms can bound autonomous work without requiring a fully formal system model.

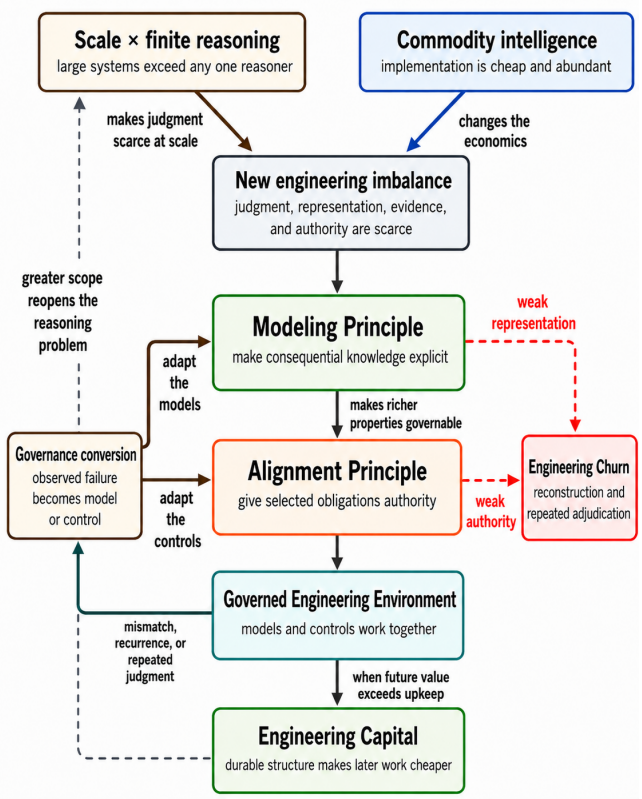

Figure 4: Conceptual relationships in MAGE. Scale creates an enduring reasoning problem, while commodity intelligence makes implementation abundant relative to engineering judgment. Modeling makes consequential knowledge explicit; Alignment gives selected obligations authority; and together they form a governed engineering environment. Governance conversion turns observed failures into models or controls that later work can inherit. Weak representation or authority instead produces engineering churn.

6.1 Engineering Questions → Governed Work

MAGE begins with an engineering question: what to learn, change, preserve, or make true. The engineer externalizes the smallest model needed to answer that question and assigns proportionate authority to settled, consequential obligations. The starting move depends on two conditions: how much implementation already exists and how settled the relevant intent is (Table 2).

The matrix is diagnostic rather than sequential. Different subsystems may occupy different cells, and a project may move between cells as implementation accumulates or intent stabilizes. After the starting move, the same cycle applies: model the reduction that makes the question answerable, align settled obligations with appropriate evidence and authority, perform the work, convert recurring reconstruction or judgment into reusable structure when its future value justifies the upkeep, and reconcile drift. The cycle does not require a comprehensive system model; structural, behavioral,

Table 2: Starting matrix for applying MAGE. Existing system stock determines what can be recovered; settled intent determines how much representation and authority the work can initially support.

	Intent uncertain	Intent settled
Little implementation	*Explore.* Build enough to learn while preserving known constraints.	*Model early.* Represent known contracts and consequential boundaries before realization.
Substantial implementation	*Recover, then explore.* Surface only enough latent structure to test competing interpretations.	*Recover, reconcile, govern.* Connect representations to the realized system before giving them authority.

ownership, decision, measurement, and provenance views can be connected selectively around the engineering question.

6.2 Application Across Project States

Greenfield projects. When little implementation exists and intent is uncertain, provisional models and inexpensive implementation support exploration, testing, and refinement. As intent stabilizes, established contracts and consequential boundaries can be modeled before realization and used to guide implementation. A written hypothesis does not create authority on its own; an obligation becomes authoritative only when its meaning is sufficiently settled and there is adequate evidence to evaluate compliance.

Brownfield projects. When substantial implementation exists, the first task is to distinguish what the system does from what the team believes it should do. Under uncertain intent, recover only the structure needed to answer the immediate question and test competing interpretations. Under settled intent, reconcile the model with the realized system before applying enforcement. A bounded migration can use *Audit* → *Drain* → *Promote* to measure the gap, reduce the remaining violations, and then authorize the obligation.

Application across engineering scales. The same method can be applied by an individual, team, product, or organization. An individual can model and govern the surface under active change; a team can connect local models to shared architecture, ownership, and operational knowledge; a product can connect representations and obligations across engineering surfaces; and an organization can carry selected policies, assurance obligations, and engineering knowledge across products. The scope and ownership of representations change, but the underlying questions remain: what should be represented, what evidence is sufficient, which obligations warrant authority, and which decisions still require judgment. Stable identities and explicit correspondence allow models at different scales to inform and constrain one another without collapsing into a single organization-wide model [5].

Scale and assurance strength. Adoption scope and assurance strength are separate choices. An individual may use formal verification for a consequential protocol, while an organization-wide deployment may rely primarily on design records, tests, runtime evidence, and human admission decisions. MAGE spans a range from lightweight representations of intent and selective checks to formal specifications and trustworthy derivation. The appropriate level depends on consequence, decidability, and maintenance cost.

6.3 Illustrations of the Method

Why not just prompt? At low implementation volume, engineers can compensate for incomplete instructions through review and explanation. At agentic velocity, this compensation becomes an assurance bottleneck. Prompts, context, and skills can guide work, but they do not by themselves establish correspondence, provide independent acceptance evidence, or make an obligation binding. A rule remains guidance unless a constraint, validator, gate, or accountable admission decision gives it operational consequence [5].

Partial automation. Spec-driven development (SDD) shows how parts of this workflow can be proceduralized. Kiro externalizes requirements, design, and implementations while retaining human approval at specification boundaries; GitHub Spec Kit provides a related artifact-centered workflow [9, 14]. These systems combine forms of Modeling and Alignment, but their specifications require correspondence with the realized system and evaluation.

As part of DocAble, we developed a Claude skill called "self-governance" that packages the Modeling–Alignment loop as a procedure an agent can apply to its own work. A recurring Claude Code hook prompts the agent to identify modeling opportunities and missing alignment, allowing it to refine its governed environment. In our use, it reliably proposes local refinements but rarely major changes. Thus, partial automation shifts human effort toward consequential judgment rather than eliminating it.

7 Research Agenda

We present an initial research agenda for empirically testing selected claims of MAGE. We focus here on three hypotheses:

H1—Representation leverage. A task-relevant model will reduce reconstruction cost or increase task scale at matched quality relative to implementation-level context alone; the advantage will grow with reasoning burden.

H2—Representation integrity. Stale, irrelevant, or poorly chosen models will reduce or reverse H1's benefit, and correspondence or obligation-coverage failures will predict later churn or defect escape beyond implementation-level metrics.

H3—Representation-induced determinization. For selected system-level obligations, an explicit model will make a repeatable predicate feasible where evaluation over raw implementation otherwise requires semantic reconstruction.

7.1 Experimental Designs

Environment-by-agent experiment. A first study would compare reasoning engines across otherwise comparable environments with different levels of representation support. Tasks would span local implementation, cross-component changes, behavioral reconstruction, and architectural work. Primary outcomes would include task quality, reconstruction cost, and task scope. The study would test whether representation quality interacts with reasoning burden and agent capability, as predicted by H1.

Our preliminary measurements provide initial evidence in support of this claim. Within DocAble, we conducted a two-week *in situ* experiment in which agents were randomly assigned to receive either explicit instructions for using the models or no such

instructions. Bidirectional linkages between code and models ensured that all agents had access to and used the models. Agents receiving explicit instructions nevertheless required fewer tokens, turns, and elapsed time to complete their tasks. The effect was more pronounced for the weaker model (Sonnet).

Representation-integrity intervention. A second study would compare synchronized, stale, irrelevant, and absent representations in matched tasks or subsystems. The study would measure correspondence, obligation coverage, reconstruction cost, churn, and defect escape. This design would test H2 and identify the conditions under which representations cease to provide leverage.

Representation-induced determinization study. A third study would select system-level obligations for which evaluation currently requires semantic reconstruction. For each obligation, researchers would introduce an explicit model and corresponding predicate, then compare evaluation with and without the model. The primary outcome would be whether the obligation becomes repeatedly evaluable without reconstructing its semantics from the implementation. This study would directly test H3.

7.2 Measures and Falsification

Across these studies, evaluation will focus on engineering outcomes. Core measures include reconstruction cost, task quality, task scope, correspondence, obligation coverage, churn, defect escape, and model maintenance cost.

The hypotheses are explicitly falsifiable. H1 is weakened if trustworthy representations do not reduce reconstruction cost or increase task scope. H2 is weakened if representation integrity does not predict downstream failures or if stale and irrelevant representations do not reduce the benefits of H1. H3 is weakened if explicit models do not enable repeatable predicates or if evaluation remains equally dependent on semantic reconstruction. Together, these studies will establish when representations provide durable engineering leverage, when they become liabilities, and when they can convert system-level knowledge into repeatable evaluation.

8 Discussion

Effect size and experimentation. Software engineering has long sought theories that explain how engineering practices affect software outcomes. One reason this has been difficult is that, historically, the implementation substrate (the code) was not easily separated from the engineers manipulating it. Differences among developers, *e.g.*, in expertise and motivation, confounded experiments: human factors often dominate the observable effects of the engineering mechanisms under study.

MAGE argues that as implementation capacity becomes abundant, the engineered environment becomes increasingly consequential to realized performance. At the same time, commodity agents make implementation capacity more separable from that environment than human engineering historically allowed. Hence, compared to many theories of software engineering, MAGE theory has more experimentally separable constructs and is thus more amenable to empirical study.

The same increase in capacity that makes the effect larger also helps expose its mechanisms. Weak representations, missing constraints, inadequate evidence, and expensive human checkpoints that were tolerable at human implementation rates should increasingly register as bottlenecks as implementation becomes cheap. Conversely, environments that successfully externalize consequential knowledge and give obligations effective authority should convert more of that capacity into durable progress. Agentic software engineering may therefore offer something beyond a new phenomenon for software engineering research to explain: it may make longstanding questions about the contribution of engineering structure easier to study. MAGE proposes one theory of those relationships; its value will depend on whether its predicted mechanisms and directional effects survive controlled variation across agents, environments, tasks, and organizations.

Classical formal methods are one instantiation of MAGE. MAGE overlaps with formal methods in using explicit representations to support assurance. MAGE characterizes the broader engineering pattern, including heterogeneous representations and enforcement mechanisms. Within this pattern, interactive proof assistants [3] provide a concretization of Modeling–Alignment at one end of the spectrum, favoring formal unification and trustworthy derivation. MAGE's flexibility is deliberate, accommodating the ambiguity, rapid change, and evolution of ordinary software engineering where grounding everything in one formal logic may not be practical.

9 Conclusion

In this paper, we propose Model-Based Agentic Software Engineering (MAGE), a theory of trustworthy agentic software engineering centered on the governed engineering environment. MAGE distinguishes two principles: Modeling makes consequential knowledge and intent explicit in task-relevant representations, while Alignment gives selected obligations authority through constraints, sensors, validators, and gates. We developed the theory from the longitudinal DocAble case and refined it through comparative reconstructions of six independently reported industrial systems.

MAGE is intended as a theoretical framework and research agenda for trustworthy agentic software engineering. It advances the hypothesis that appropriately engineered representations can reduce the reconstruction required of autonomous agents and enable more durable engineering progress, while alignment mechanisms can translate selected engineering obligations into enforceable action boundaries. Testing these claims requires systematic study of how representations, controls, and human decision boundaries shape agent behavior across tasks, projects, and environments.

References

[1] Adem Ait, Gwendal Jouneaux, Javier Luis Cánovas Izquierdo, and Jordi Cabot. 2025. Towards Automated Governance: A DSL for Human-Agent Collaboration in Software Projects. arXiv:2510.14465 [cs.SE] https://arxiv.org/abs/2510.14465

[2] Daniel Berger. 2026. Your Next Colleague Is an Agent: The Dawn of Agentic AI-Aided Engineering. Siemens Digital Industries Software Blog. https://blogs.sw.siemens.com/art-of-the-possible/agentic-ai-aided-engineering/ Accessed 20 August 2026.

[3] Adam Chlipala. 2026. Automatic Programming Should Be More Like SQL: How to Raise the Abstraction Level, Reliably. Structure and Guarantees. https://stng.substack.com/p/automatic-programming-should-be-more Published 18 August 2026; accessed 21 August 2026.

[4] Edmund M. Clarke, Orna Grumberg, and Doron A. Peled. 1999. *Model Checking*. MIT Press.

[5] James C. Davis. 2026. *The MAGE Method: Model-Based Agentic Software Engineering*. Unpublished manuscript. Working manuscript, edition last modified 20 August 2026.

[6] James C. Davis, Paschal C. Amusuo, Tanmay Singla, Berk Çakar, and Kirsten A. Davis. 2026. Cheap Code, Costly Judgment: A Case Study on Governable Agentic Software Engineering. arXiv:2607.01087 [cs.SE]

[7] Manuel de la Peña. 2026. A Virtual Agent Team at Docker: How the Coding Agent Sandboxes Team Uses a Fleet of Agents to Ship Faster. Docker Blog. https://www.docker.com/blog/a-virtual-agent-team-at-docker-how-the-coding-agent-sandboxes-team-uses-a-fleet-of-agents-to-ship-faster/ Accessed 20 August 2026.

[8] Philip Dufwa and Thomas Luvö. 2026. A Platform for Scalable Enterprise AI Agents. Zenseact. https://zenseact.com/news/a-platform-for-scalable-enterprise-ai-agents/ Accessed 20 August 2026.

[9] GitHub. 2025. Spec-Driven Development with AI: Get Started with a New Open Source Toolkit. GitHub Blog. https://github.blog/ai-and-ml/generative-ai/spec-driven-development-with-ai-get-started-with-a-new-open-source-toolkit/ Accessed 2 September 2025.

[10] Hao He, Courtney Miller, Shyam Agarwal, Christian Kästner, and Bogdan Vasilescu. 2026. Speed at the Cost of Quality: How Cursor AI Increases Short-Term Velocity and Long-Term Complexity in Open-Source Projects. In *Proceedings of the International Conference on Mining Software Repositories (MSR)*.

[11] Yuyang Hu, Shichun Liu, Yanwei Yue, Guibin Zhang, Boyang Liu, Fangyi Zhu, Jiahang Lin, Honglin Guo, Shihan Dou, Zhiheng Xi, Senjie Jin, Jiejun Tan, Yanbin Yin, Jiongnan Liu, Zeyu Zhang, Zhongxiang Sun, Yutao Zhu, Hao Sun, Boci Peng, Zhenrong Cheng, Xuanbo Fan, Jiaxin Guo, Xinlei Yu, Zhenhong Zhou, Zewen Hu, Jiahao Huo, Junhao Wang, Yuwei Niu, Yu Wang, Zhenfei Yin, Xiaobin Hu, Yue Liao, Qiankun Li, Kun Wang, Wangchunshu Zhou, Yixin Liu, Dawei Cheng, Qi Zhang, Tao Gui, Shirui Pan, Yan Zhang, Philip Torr, Zhicheng Dou, Ji-Rong Wen, Xuanjing Huang, Yu-Gang Jiang, and Shuicheng Yan. 2026. Memory in the Age of AI Agents. arXiv:2512.13564 [cs.CL]

[12] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-world Github Issues?. In *The Twelfth International Conference on Learning Representations*.

[13] Maurits Kaptein, Vassilis-Javed Khan, and Andriy Podstavnychy. 2026. Runtime Governance for AI Agents: Policies on Paths. arXiv:2603.16586 [cs.AI]

[14] Kiro. 2026. Specs. Kiro Documentation. https://kiro.dev/docs/specs/ Accessed 20 August 2026.

[15] Christopher Koch. 2026. From Governance Norms to Enforceable Controls: A Layered Translation Method for Runtime Guardrails in Agentic AI. arXiv:2604.05229 [cs.AI]

[16] Paul M Leonardi. 2011. When flexible routines meet flexible technologies: Affordance, constraint, and the imbrication of human and material agencies1. *MIS quarterly* 35, 1 (2011), 147–167.

[17] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Neural Information Processing Systems*.

[18] Martin Monperrus. 2026. The End of Code Review: Coding Agents Supersede Human Inspection. arXiv:2606.13175 [cs.SE]

[19] Javier Moreno and River. 2026. Under the River. Shopify Engineering. https://shopify.engineering/under-the-river Accessed 20 August 2026.

[20] Wanda J Orlikowski. 2000. Using technology and constituting structures: A practice lens for studying technology in organizations. *Organization science* 11, 4 (2000), 404–428.

[21] Huiyun Peng, Arjun Gupte, Ryan Hasler, Nicholas John Eliopoulos, Chien-Chou Ho, Rishi Mantri, Leo Deng, Konstantin Läufer, George K. Thiruvathukal, and James C. Davis. 2026. SysLLMatic: Large Language Models are Software System Optimizers. *Journal of Systems and Software* 240 (2026), 112929. doi:10.1016/j.jss.2026.112929

[22] Huiyun Peng, Parth Vinod Patil, Antonio Zhong Qiu, George K. Thiruvathukal, and James C. Davis. 2026. Beyond Local Code Optimization: Multi-Agent Reasoning for Software System Optimization. In *Journal Ahead Workshop (JAWs), co-located with the IEEE/ACM International Conference on Software Engineering (ICSE)*.

[23] Huiyun Peng, Antonio Zhong Qiu, Ricardo Andrés Calvo Méndez, Kelechi G. Kalu, and James C. Davis. 2026. How Do Agents Perform Code Optimization? An Empirical Study. In *Proceedings of the 23rd International Conference on Mining Software Repositories (MSR '26)*. Association for Computing Machinery, New York, NY, USA, 732–736. doi:10.1145/3793302.3793564

[24] Benjamin C. Pierce. 2002. *Types and Programming Languages*. MIT Press.

[25] Timo Reimann. 2026. How Cloudflare Enforces Engineering Standards Using AI. Cloudflare Blog. https://blog.cloudflare.com/engineering-standards-enforcement/ Accessed 20 August 2026.

[26] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[27] Douglas C. Schmidt. 2006. Model-Driven Engineering. *Computer* 39, 2 (2006), 25–31.

[28] Spotify Engineering. 2026. Coding Is No Longer the Constraint: Scaling Developer Experience to Teams and Agents at Spotify. Spotify Engineering. https://engineering.atspotify.com/2026/6/code-with-claude-coding-is-no-longer-the-constraint Accessed 20 August 2026.

[29] U.S. Department of Justice. 2024. Nondiscrimination on the Basis of Disability; Accessibility of Web Information and Services of State and Local Government Entities. *Federal Register* 89, 82 (April 2024), 31320–31396.

[30] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei, and Jirong Wen. 2024. A survey on large language model based autonomous agents. *Frontiers of Computer Science* 18, 6 (March 2024). doi:10.1007/s11704-024-40231-1

[31] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. 2025. OpenHands: An Open Platform for AI Software Developers as Generalist Agents. In *International Conference on Learning Representations (ICLR)*.

[32] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[33] Titus Winters, Tom Manshreck, and Hyrum Wright. 2020. *Software Engineering at Google: Lessons Learned from Programming Over Time*. O'Reilly Media.

[34] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[35] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations*.